Automatic discovery of cyberattacks

Guillaume Hutzler*, Hanna Klaudel*, Witold Klaudel†‡, Franck Pommereau*, Artur Rataj†
* IBISC, Univ. Evry, Université Paris-Saclay, France
Email: {guillaume.hutzler, hanna.klaudel, franck.pommereau}@univ-evry.fr
† IRT SystemX, Palaiseau, France, Email: artur.rataj@irt-systemx.fr
‡ SafeTech Cybernetics, Palaiseau, France, Email: witold.klaudel@outlook.fr

***Abstract*—We present a method for an automatic discovery of cyberattacks in a distributed information system. The method creates a formal model of branching propagation of the attacker from the specification of software and hardware architecture enriched with multi-faceted access control information.**

The ensuing network of automata, decorated with unitary attack costs, enables a vast range of model checking techniques, in particular the search for the most efficient attack strategies.

The model is expressive enough to cover a wide range of attack and defence approaches, like false data injection or redundant data sources.

***Index Terms*—security evaluation, formal modelling**

I. Introduction

Identifying threats to distributed information systems is challenging as they consist of many different entities often deployed in safety-critical sectors exposed to a wide range of cyberattacks. A cyberattack is understood here as a sequence of (potentially branching) unitary actions taken by an *attacker* to take control over some components of the system. This sequence starts with one or more entry points and ends with the loss of integrity of some system components leading in turn to a damage targeted by the attacker. Identifying cyberattacks is crucial for optimising both an architecture and cybersecurity features to meet an acceptable level of risk. It is necessary to calculate likelihood of cyberattacks and thus contribute to the estimation of the overall risk level. The latter depends both on the commercial impacts of the damage targeted by the attackers and on the likelihood of the attacks.

When it comes to interpreting the concept of likelihood, many national cybersecurity agencies (e.g., NIST [1], ANSSI [2]) issue loosely defined recommendations and defer final decisions for its rating scale and methods to experts. The choice of scale and methods for assessing the strength of cyber-protections is still an open research problem, *e.g.* [3]. In this work, we lean towards the ANSSI approach and thus, translate the likelihood into an inversely proportional cost. This allows us to calculate the cost of the entire attack as a sum of costs of all its unitary attacks. We also assume that experts can provide the scale and value of costs associated with breaking the cyber-protections of the systems under analysis.

In most approaches, a formalism of attack trees [4], [5], [6] (or variants like forests [7] or graphs [8], [9], [10], [11], [12]) is used to describe ways attacks can happen. Edges of an attack tree correspond to unitary attacks allowing the attacker to put the system into a desired insecure state. Attack trees offer the possibility of some quantitative analysis regarding costs, probabilities or likelihood, globally or for individual sub-attacks [13]. This allows them to be combined with cybersecurity features [14], [6] which can then be optimally adjusted to meet an acceptable level of risk.

The task of creating an attack tree is often assigned to security specialists and remains manual. Since the manual creation of the attack tree is tedious and error-prone, several authors propose to generate it automatically [15], [7], [16], [17] [18], [19], [20], [9], [11]. As the basis for the attack tree generation, the authors choose a set of predefined unitary attacks (often called *atomic actions*) that the attacker can perform to put the system into an insecure state. The chosen set corresponds to known vulnerabilities present in the system or to zero-day (unknown) flaws that the expert decides to consider. The languages proposed to describe atomic actions make it possible to define system state variables and attack propagation rules. Each action has pre-conditions, including sometimes network reachability, and post-processing impacting the state of the system. The evolution of the system state represents the attacker's progress towards the attack target.

All authors cited above assume the analysed system to be secure except for flaws explicitly introduced by atomic actions with specific propagation rules defined by experts. Functional and technical relationships between components of the analysed system are not used directly but are only reinterpreted by experts when defining atomic actions. For these reasons, these approaches are mostly suitable for audits of already existing systems but are less convenient for the design phase and the selection of the best choice of cybersecurity features for a system under construction.

In this paper, we propose an approach for the modelling of cyberattack propagation in distributed information systems. Unlike many existing approaches, it does not rely on a predefined set of known vulnerabilities; instead it considers that each component can be compromised at a cost to attackers depending on the cybersecurity protections. This allows the discovery of potential attacks due to the choice of the strength of cybersecurity protections within the components constituting the analysed system. Our method facilitates the reuse of existing models by adapting them to the needs of various life cycle phases of the analysed system, either by modifying the costs of breaking security barriers (*e.g.* to consider freshly published vulnerabilities) or by adapting the architecture to new configurations (*e.g.* adding a redundant sensor).

The approach allows an automatic estimation of cyberattack

likelihood, which subsumes most attack schemes used in attack tree approaches. It introduces a generic method for an automatic generation of an attack propagation model in a way which is similar, to a certain degree, to that initiated by Ritchey and Ammann [12]. The propagation model is derived from both the description of the software and hardware layers of the analysed system and from a set of generic unitary attack propagation rules associated with classes of software components and communication protocols. It forms a set (network) of automata, each of them representing a software component with its knowledge of shared secrets (stolen or not) and status (functional, non-available, bad-data, or malware). The attacker's presence in the model is represented as a status different from functional of some automata. Each automaton evolves according to a set of attack propagation rules. As the model forms a set of automata, an off-the-shelf model checker can be used to find temporal logic properties describing the evolution of an attack from its initial state indicating the attacker's original entry.

The definition of automata constitutes an innovative aspect of our approach, as they result from our interpretation of the loss of classical cybersecurity properties such as integrity, availability, and confidentiality. Losing integrity by a component leads to one of two possible states: bad-data, when the component produces incorrect data and its propagation capabilities are reduced, or malware, when the component gains very strong propagation capabilities including impersonation of all other states. The loss of availability leads to non-available state, while that of confidentiality results in disclosure of confidential information (secret theft).

It is important to emphasise that our modelling approach is dedicated to representing the propagation of the attack and does not describe the operational behaviour of the system. The state of the system is therefore devoid of variables describing its physical state, such as temperature or speed for example. The only exception was made for informational redundancy, which is taken into account because it is considered as one of the cyber-defense features.

The architecture description language used for model generation is not in the scope of the present paper. It allows to express most cybersecurity features present in typical distributed information systems such as: routing restrictions to isolate subsystems, operating systems enabling secure computing resource sharing, application access control, security of network interactions to protect application resources. It is enriched with additional concepts such as informational redundancy or security keys and attack propagation patterns.

In the remainder of the paper we give an overview of our method in Section II and its main elements are illustrated using a toy example. In particular the key notions of visibility graph, propagation rules and attacker's positions in the system are defined and used to construct a network of automata. A more real-life illustrative case study is presented in Section III as well as attack scenarios discovery using model-checking. Section IV discusses outcomes and limitations. Finally, in Section V we give some concluding remarks and perspectives for the future work.

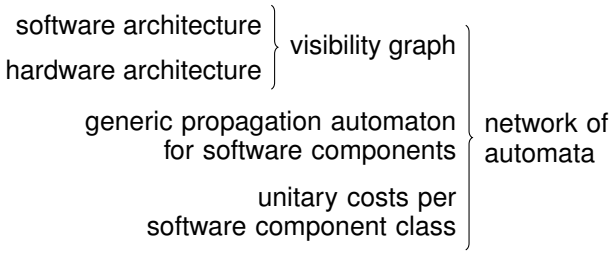

Fig. 1. Model construction elements and dependencies

II. Our modelling approach

We describe the system as an architecture consisting of the hardware and software and the respective mapping. Each software component exposes interfaces which are necessary for its normal functioning but can be abused by an attacker to penetrate the system. Interface exposure depends on functional complexity of the system but also on hardware architecture constraints such as resource sharing and network routing.

We assume that an attacker can pass through any interface but with different costs. The definition of these costs is an important part of the system specification that is used to generate the model to be queried to search for attacks. Attack discovery corresponds in this context to the search for sequences of interface crossings leading from the initial state of the system with one or more attackers positioned in software components to certain (other) undesirable target states. Among the huge number of possible attacks, we select, using model checking queries, those having a minimal cost, or costs under some fixed bound. This bound corresponds to the maximum likelihood needed to maintain an acceptable level of risk.

As shown in Fig. 1, a model generation goes through an intermediate construction called a *visibility graph*, which results from the synthesis of the software and hardware architectures and focuses only on the information flow allowing propagation of the attacker in the system. To obtain the queryable model, the visibility graph is completed by the definition of a generic automaton specifying the behaviour of a software component under attack and the corresponding costs.

A. Architecture

Hardware components are connected by undirected network links while software components are connected by directed *functional interactions* encapsulated in communication protocols, *e.g.* HTTP for web applications. Software interaction endpoints are attached to software components through software ports called *roles*, *e.g.* client and server for HTTP protocol. Software components are hosted by hardware components and functional interactions are routed through network links. We assume that each software component is assigned to exactly one hardware component. A privileged software component (kernel) may also manage its hardware host, including subordinate software components within the same hardware host and network or intra-system routing rules.

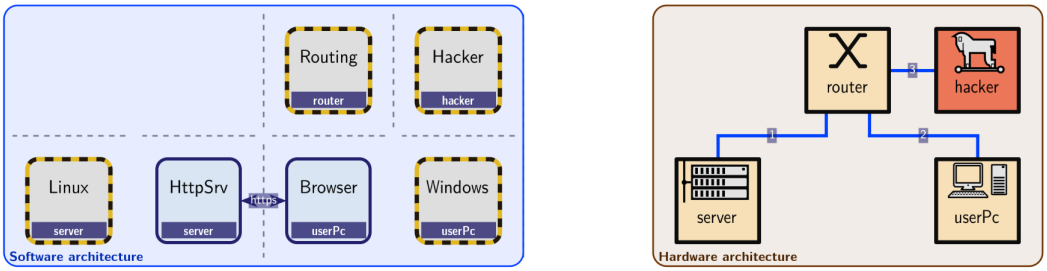

Fig. 2. Toy example - architecture.

The hardware link types correspond to the communication link types such as for example, Ethernet or CAN network. Hardware links are connected to hardware components through hardware ports. Hardware components are managed by operating systems (kernels). Each hardware component has its own *routing rules*, which determine the physical paths of interactions. These rules are quite general: they define sets of allowed interactions, which can be routed through hardware ports to software components (software components interacting with one another through the network), from one hardware port to another (hardware component acting as a network router), and also between software components sharing the same hardware (inter-process interactions between software components sharing the same hardware).

Routing rules must ensure a physical realisation of all specified functional interactions. Often, due to technical and organisational constraints, routing rules are more permissive than necessary, which can result in the creation of additional attack opportunities.

Example 1 (Toy example - architecture): We consider a system with an HTTP server and a user browser interacting with it, as in Fig. 2. The hardware component server hosts two software components: HttpSrv for HTTP functionalities and Linux as operating system managing server. The userPc hardware hosts the Browser and Windows operating system. The router hardware only hosts the Routing firmware (with operating system privileges). The detailed software architecture of the hacker is not in the scope of our interest and is modelled as a unique software component Hacker seen as an operating system.

The Browser and the HttpSrv communicate through a router. The router allows any component to communicate bidirectionally using HTTPS protocol with the HttpSrv but does not accept any other traffic. The server and the userPC limit external communications to HTTPS interactions. The intruder (hacker), connected to the router, will try to compromise the system. ◊

B. Visibility graph

The visibility graph is a directed graph indicating which software component (node) can propagate its corrupted status to its successors. In order to define edges, let us introduce the auxiliary concepts of *implicit* interactions, and *system role*, which are used in the construction.

Implicit interactions relate to the sharing of hardware components and network links by software components and from the dependency of software components on operating systems managing hardware ones. Functional interactions are specified explicitly in the model, while implicit interactions are added automatically via templates following a chosen interpretation, like *e.g.* that of a typical operating system with respect to the applications under its management, or that of an application which attacks an other application through a functional role of the latter but which is not in the functional scope of the former.

System roles, subdivided into kernel-network, kernel-application, and application-kernel roles, relate to the interfaces exposed by kernels[1] to their applications or to network interfaces. All of them can be used to create implicit interactions, provided they are authorised by the routing rules.

The edges of the visibility graph are obtained from all functional and implicit interactions. Each interaction is directed and gives rise to an edge connecting a node to the role of its target node, while the source role is not indicated. Edges are labelled by the *attacker position* relative to this interaction:

- `peer` if the attacking node is a functional peer within that interaction or if the edge points to a system role;
- `mitm` (man in the middle) if the attacking node is on a routing path taken by the interaction;
- `side` if the routing rules merely allow the attacking node to see the target role but the node is not in `peer` or in `mitm` positions.

For each functional role, there are two types of edges: an edge for any functional interaction, *i.e.* from any functional partner of such interaction, and an edge from any kernel-network role or any other functional one. For each kernel-network role, there is an edge from any functional or kernel-network role. Edges between kernel-application and application-kernel roles, in both directions, exist for kernels and the applications they manage. Once the visibility graph is created, the source roles of edges are no more pertinent, and are replaced by software components.

Definition 1: A *visibility graph* is a tuple $VG = (N, R, E)$ where

- N is the set of nodes (software components);

[1]At this level of abstraction, we do not distinguish between kernels and operating systems and use indifferently both names.

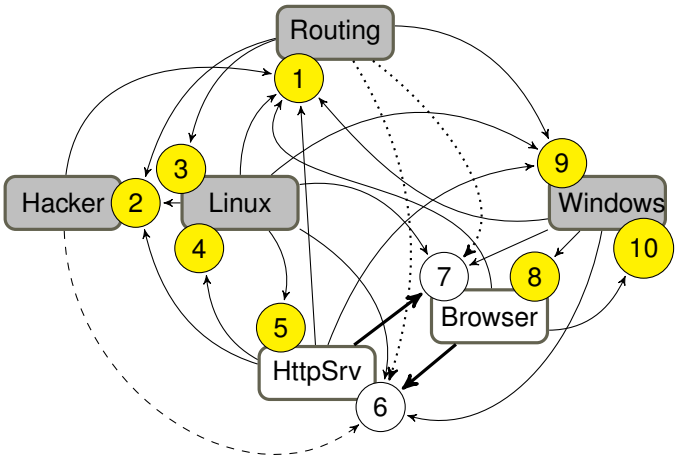

Fig. 3. The visibility graph of the toy example. Edges labelled `mitm` are dotted, `side` are dashed, and `peer` are solid thin. Functional edges are solid thick.

- R is a set of *roles* of all the nodes; $R = R_1 \cup \ldots R_{|N|}$, where all R_n are pairwise disjoint and each role has a *type* (system or functional);
- $E \subseteq N \times R$ is the set of edges, each of them being labeled by the attacker position: `peer`, `mitm` or `side`; $E = E_{\texttt{peer}} \cup E_{\texttt{mitm}} \cup E_{\texttt{side}}$. ◊

Example 2 (Toy example - visibility graph): Fig. 3 presents the visibility graph of the toy example. Kernel components are depicted in gray, functional ones in white, functional edges are indicated with thick lines, implicit ones with thin lines, functional roles are white, and the three kinds of kernel roles, *i.e.* kernel-network, kernel-application, and application-kernel are all in yellow.

Routing is a central point of the system since the routing firmware, when compromised, can attack the entire system. Routing has thus visibility on all roles of all software components except the kernel-application ones (4,5,8,10): it can attack Hacker, Linux and Windows through their kernel-network roles (3,2,9), and HttpSrv and Browser through their functional roles (6,7). Edges from Routing to functional roles (6,7) are labeled `mitm` because the network path of bidirectional interaction between HttpSrv and Browser passes through Routing.

Due to restricted routing rules, the Hacker's visibility is limited to the functional role of HttpSrv (6) and the kernel-network role of Routing (1). Its attack position for role 6 is `side` because it is neither a partner nor on the path of the functional interaction between Browser and HttpSrv. ◊

C. Attack propagation specification and costs

Given a visibility graph and some auxiliary information such as thresholds, secrets and roles categories, we are able to define the model of the system, which is a network of automata $\{A_1, \ldots, A_{|N|}\}$ (one for each node of the visibility graph), which have identical form and differ only by the parameters in the transition conditions (guards). As they have identical form, we shall define only one generic automaton, and each copy of it will have, both initially and during the system execution, its own current state in $L = \{\mathcal{F}, \mathcal{N}, \mathcal{B}, \mathcal{M}\}$ meaning respectively functional, non-available, bad-data or malware. A node n will be considered to be *active* if its automaton is in a state different from $\mathcal{N}$.

We define the model in several steps:

- first, we define an *attack propagation specification* AP (Def. 2), which encompasses the visibility graph and additional specific information such as secrets, thresholds, and roles categories;
- a *configuration* of AP (Def. 3) is the state of the automata of its nodes and the state of the secrets, which may evolve during the execution. The state of the automaton of a given node is called its *status* and is initially given by the configuration;
- the *generic automaton* (Def. 4) has transitions and updates that depend on AP and its configuration, firing a transition yielding a new configuration;
- the firing of transitions is equipped with a *cost function* (Def. 5) that evaluates the cost for the attacker to perform a transition;
- the *semantics* (Def. 6) defines how firing of transitions from an initial configuration allows to build executions gathered in a state-space.

Definition 2: An *Attack Propagation Specification* is a tuple $AP = (VG, \texttt{cat}, \texttt{thresh}, \texttt{open}, S)$ where:

- $VG = (N, R, E)$ is a visibility graph,
- $\texttt{cat}\colon R \rightarrow \{\texttt{man}, \texttt{opt}, \texttt{transp}\}$ is the role *category*, which may be resp. mandatory, optional or transparent;[2]
- `thresh` encompasses two thresholds:
  - $\texttt{thresh}_{\mathcal{N}}\colon N \rightarrow \mathbb{N}$ is an *activity threshold* indicating, for each node, the minimum number of optional roles having active predecessors that are necessary to preserve the node's activity;
  - $\texttt{thresh}_{\mathcal{B}}\colon N \rightarrow \mathbb{N}$ is a *plausibility threshold* indicating, for each node, the minimum number of optional roles accepting plausible bad data that are necessary to fool the node's informational redundancy and possibly change its status to $\mathcal{B}$;
- $\texttt{open}\colon E \rightarrow \varphi$ associates, with each edge in E, a Boolean formula over statuses of nodes and indicates the conditions required for communication via the edge;
- S is a finite set of globally shareable *secrets* with two functions:
  - $\texttt{Store}\colon (N \times S) \rightarrow \mathbb{B}$ indicating which secrets are stored in nodes, *i.e.* from which nodes they can be stolen,
  - $\texttt{Protection}\colon (E \times S) \rightarrow \mathbb{B}$ indicating which secrets protect communication sessions represented by edges. ◊

Each edge in E in the visibility graph represents a communication session between two software components, component n in role r and its predecessor $p \in \texttt{pre}(n) = \{p \mid$

[2]System roles category is always `transp` because they do not impact the behaviour of software components.

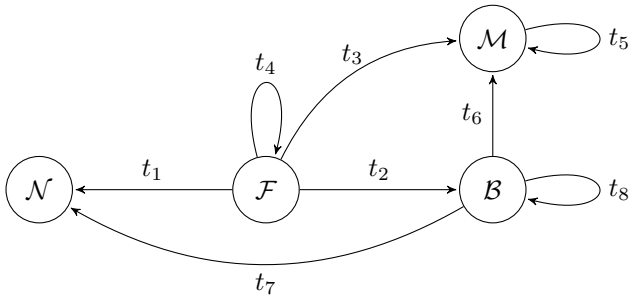

Fig. 4. General shape of the automaton A_n of node $n \in N$.

$\exists r \in R_n, (p, r) \in E\}$. In order for the session to be `open`, at least one communication routing path should be available, *i.e.* the statuses of router nodes on this path should be different from $\mathcal{N}$. This is expressed statically by a Boolean formula φ obtained from the architecture when generating the visibility graph. Only nodes belonging to $\texttt{pre}(n)$ in the `mitm` attack position can be involved in φ.

Example 3 (Toy example - `open` *formulas):* Let us look more closely at two edges in the visibility graph from Fig. 3:

- The edge from Browser to role 6 of HttpSrv corresponds to the functional interaction between software components, which is routed trough the router managed by the Routing software. To be `open`, the edge needs availability of three software components: Windows, Routing and Linux. The logical formula is then

  $$L(\text{Windows}) \neq \mathcal{N} \wedge L(\text{Routing}) \neq \mathcal{N} \wedge L(\text{Linux}) \neq \mathcal{N},$$

  where $L(.)$ denotes the status of the component.
- The edge from Browser to role 10 of Windows corresponds to the implicit interaction between Browser and its managing software Windows. In our convention, this interaction is always open and the formula is just `true`. $\Diamond$

Secrets in S refers to security keys protecting communication sessions, which can be stolen from the nodes where they are stored. Once stolen, it impacts the cost calculation by eliminating the costs of breaking the communication protocols' security of the edges (interactions) that they protect. We assume that stolen secrets are visible globally, which corresponds to an omniscient attacker that has a global knowledge of the system (or at least the parts that it controls).

Definition 3: A *configuration* of an $AP = (VG, \texttt{cat}, \texttt{thresh}, \texttt{open}, S)$ with $VG = (N, R, E)$, is a pair $(\overrightarrow{q}, \overrightarrow{s})$, where $\overrightarrow{q} \in L^{|N|}$ is the vector of states of the automata and $\overrightarrow{s} \in \mathbb{B}^{|S|}$ the current value of secrets, which may be stolen (`true`) or not (`false`). $\Diamond$

Definition 4: The behaviour of each node $n \in N$ is defined by the automaton $A_n = (L, T, \gamma, \delta)$ shown in Fig. 4, where:

- L is the set of its states;
- $T = \{t_1, \ldots, t_8\}$ is its set of transitions shown in Fig. 4,
- $\gamma \colon T \to \mathbb{B}$ is a guard on each transition and,
- $\delta \colon T \times \mathbb{B}^S \to \mathbb{B}^S$ is an update of secrets' global knowledge. $\Diamond$

Intuitively, the status change of node n depends on its own status and on the status of its predecessors $p \in \texttt{pre}(n)$. More precisely, the impact of p may vary depending on the role r of n to which p is connected. It depends on the category and type of r, and on the attacker position labelling the edge from p to r.

From the attacked node's role perspective, the status of its predecessor may be interpreted as functional data (i_1), non-available data (i_2), bad plausible data (i_3), bad non-plausible data (i_4), non-operational code injection (i_5), fully operational code injection (i_6), bad data malware code injection (i_7), remote steal of secret keys (i_8), or not considered (i_9); the latter occurring if the predecessor of n is not able to impact the status of the attacked node. Bad plausible data is understood as a bad data which is accepted as correct by the role plausibility check. Each of these interpretations leads to a unitary cost participating in the calculation of the total cost of the attacked node status change. In this respect, two cases have to be addressed: either the predecessor of node n can communicate with n (the `open` formula is true) or not (the `open` formula is false). If the `open` formula evaluates to `false`, only predecessors of n in the `peer` position are considered and are always interpreted as non available data. If the `open` formula evaluates to `true`, the following cases exist:

- status $\mathcal{F}$ is always interpreted as functional data (i_1);
- status $\mathcal{N}$ is always interpreted as non-available data (i_2) if the predecessor attacker position is `peer`, otherwise it is not considered (i_9);
- status $\mathcal{B}$ may be one of the following if the attacker position is `peer`:
  - bad plausible data (i_3) or bad non-plausible data (i_4), or,
  - one of three types of malware code injection (i_5, i_6, i_7),

  otherwise it is not considered (i_9);
- status $\mathcal{M}$ in any attacker position can give any of the above interpretations and a supplementary one: remote steal of secret keys.

Let $I = \{i_1, \ldots, i_9\}$ be the set of interpretations defined above and $J_n = \{(p, r, i) \mid p \in \texttt{pre}(n), r \in R_n, i \in I\}$ the set of all possible *contributions* to n, *i.e.* the set of all possible ways the predecessor nodes may impact the status of n.

We can define then the status change conditions in the automaton of n, *i.e.* the transitions' guards γ and secrets' updates δ, as follows:

1) Transitions t_1 and t_7 passing the status of n to $\mathcal{N}$ are enabled if:
   - there is at least a contribution $(p, r, i) \in J_n$ with $i \in \{i_2, i_4\}$ and r a mandatory role of n, or,
   - there is a subset of contributions $J' \subseteq J_n$ and a set $R' \subseteq R_o \subseteq R_n$, where R_o is the set of all optional roles of n, such that $J' = \{(p, r, i) \mid r \in R', i \in \{i_2, i_4\}, |R_o \setminus R'| < \texttt{thresh}_{\mathcal{N}}(n)\}$, or,
   - there is a contribution $(p, r, i) \in J_n$ with $i = i_5$ meaning that the attacker intends to inject a re-

stricted malicious code forcing the status $\mathcal{N}$ for n.

2) Transition t_2 passing the status of n to $\mathcal{B}$ is enabled if:
   - if there is a non-empty subset $J' \subseteq J_n$ for which the conditions for passing of n to status $\mathcal{N}$ are not fulfilled and:
     - there is at least one $(p, r, i) \in J'$ where $i = i_3$ and r is a mandatory role of n, or,
     - there is non-empty $J'' \subseteq J'$ and $R' \subseteq R_n$ of optional roles of n such that $J'' = \{(p, r, i) \mid r \in R' \text{ and } |R'| \geq \texttt{thresh}_{\mathcal{B}}(n)\}$, or,
   - if there is $(p, r, i) \in J'$ with $i = i_7$ meaning that the attacker intends to inject a restricted malicious code forcing the $\mathcal{B}$ status for n;
3) Transitions t_3 and t_6 passing the status of n to $\mathcal{M}$ are enabled if there is at least one $(p, r, i) \in J_n$ with $i = i_6$ meaning that the attacker intends to inject a fully operational malicious code to node n.
4) Transitions t_4 and t_8 are enabled if there are locally stored secrets not already stolen and if there exists at least one $(p, r, i) \in J_n$ with $i = i_8$; the secrets are updated.
5) Transition t_5 is enabled if there are locally stored secrets not already stolen. It is the only transition that does not rely on interpretations; the secrets are updated.

D. Costs

We assume that the unitary costs are defined by experts for any contribution in J_n for any $n \in N$.

The triggering of transitions in the automata comes with costs, which encompass the following aspects:

- $\kappa_E \colon (E \times \{\texttt{theft}, \texttt{destr}\}) \to \mathbb{N}$ defines unitary costs of breaking communication protocols corresponding to edges for two types of break: session theft (`theft`) or session destruction (`destr`);
- $\kappa_R \colon ((R \times L \setminus \{\mathcal{F}\} \times \{\texttt{code}\}) \cup (R \times \{\texttt{data}, \texttt{steal}\})) \to \mathbb{N}$ defines unitary costs of breaking node's functionalities bypassing role protections. For `code`, it gives the cost of malicious code injection (i_5, i_6 and i_7) forcing the transition of the node respectively to status $\mathcal{N}$, $\mathcal{M}$ or $\mathcal{B}$. For `data` it gives the cost of producing bad plausible data, while for `steal` the cost of steal of secrets by remote malware;
- $\kappa_N \colon N \to \mathbb{N}$ defines the cost of stealing secrets by the node itself when its status is already $\mathcal{M}$ (malware).

Definition 5: The computation of costs of transitions in each automaton A_n is defined as follows:

- For each $t \in T \setminus \{t_5\}$ in the automaton of n, a set of predecessors $P \subseteq \texttt{pre}(n)$ and a choice of contributions $J \subseteq J_n$ satisfying the enabling conditions of t, the cost of t, $\kappa_{P,J}(t)$, is the sum for all $p \in P$ such that $\texttt{open}((p, n)) = \texttt{true}$ of:
  - $\kappa_E((p, r), s, \alpha)$ corresponding to the cost of breaking communication protocol between p and n on its role r, with s representing the current possession of secrets, and α being one of two possible attacks on sessions (`theft` or `destr`), and
  - $\kappa_R(r, \texttt{data})$, $\kappa_R(r, \texttt{steal})$ or $\kappa_R(r, l, \texttt{code})$ depending on one of possible attack types (`data`, `steal`, `code`) through role r, where the first is the cost of producing bad data accepted by n's plausibility check, the second is the cost of stealing secrets by remote malware and the last is the cost of malicious code injection of type (i_5, i_6 or i_7) leading to the corresponding status $l \in \{\mathcal{N}, \mathcal{M}, \mathcal{B}\}$,
- For transition t_5, $\kappa(t_5) = \kappa_N(n)$. $\Diamond$

One may observe that a transition different from t_5 in the automaton of n may be enabled for various sets of contributions $J \subseteq J_n$, each of them leading potentially to different costs, $\kappa_{P,J}$. In this case, only the lowest cost transition will be retained in the semantics of the entire system.

The second observation refers to the lack of transitions from $\mathcal{M}$ to $\mathcal{B}$ and $\mathcal{M}$ to $\mathcal{N}$, and those returning to $\mathcal{F}$. For those starting from $\mathcal{M}$, they are not considered because the set of all interpretations of $\mathcal{M}$ subsumes all those for $\mathcal{B}$ and $\mathcal{N}$, making the corresponding transitions non-interesting from the attack propagation point of view. The transitions returning to $\mathcal{F}$ are not considered as they do not activate any interpretation which could be useful for attack propagation.

E. Semantics

Definition 6: The *semantics* of an AP from an initial configuration $c_0 = (\overrightarrow{q_0}, \overrightarrow{s_0})$ is a transition system $(\texttt{Config}_{AP}, \xrightarrow{\kappa}, c_0)$ where $\texttt{Config}_{AP}$ is the set of all configurations reachable from c_0 by executing transitions defined as follows:
A transition $(\overrightarrow{q}, \overrightarrow{s}) \xrightarrow{\kappa} (\overrightarrow{q'}, \overrightarrow{s'})$ exists if

- there exists an enabled transition $t_n = (\eta, \eta')$ with the lowest cost κ in A_n, *i.e.* $\overrightarrow{q}[n] = \eta$, $\overrightarrow{q'}[n] = \eta'$, and for all $1 \leq i \leq |N|$, $i \neq n \Rightarrow \overrightarrow{q'}[i] = \overrightarrow{q}[i]$, and
- the secrets are updated, *i.e.* $\overrightarrow{s'} = \delta_n(t_n, \overrightarrow{s})$. $\Diamond$

Intuitively, the dynamics of AP only includes moves between configurations that differ in exactly one status of a node or in the increase in the number of stolen secrets. Given two configurations, at most one move is possible between them: that which corresponds to the lowest cost κ of the transition in the automaton under consideration.

F. Attack discovery

Given a propagation model, our attack discovery proceeds as follows:

- First, we define an initial configuration by positioning attackers in one or more components, *i.e.* we set the status of these components to $\mathcal{M}$;
- Second, we define an attackers' target, *i.e.* the configurations comprising some components the attackers want to force to status different from $\mathcal{F}$ or/and the secrets to be stolen;
- Finally, we use model checker and temporal logic formulas to find the attack sequence of minimal cost or to prove that such an attack sequence does not exist, at least under some fixed cost.

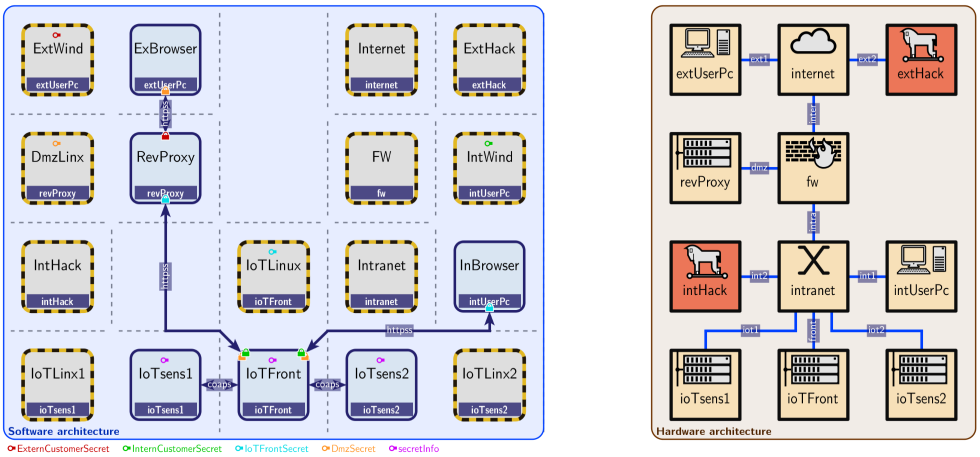

Fig. 5. Simplified architecture of the case study.

III. Illustrative Case Study

Fig. 5 depicts a system composed of an IoT server IotFront collecting data from two IoT sensors: IoTsens1 and IoTsens2. IotFront is active if at least one of the sensors is active. IotFront allows users to access collected information *via* HTTP browsers. Internal user browser (InBrowser) connects directly to IotFront while external one connects through the reverse proxy (RevProxy) located in a DMZ protected by a firewall FW. Firewall routing rules allow connectivity of all external traffic only to the reverse proxy, while Intranet routing rules are totally permissive. All HTTP sessions are protected by security keys, which allow bidirectional strong authentication of all HTTP actors. Communication protocols between sensors and IotFront are protected by COAP protocol and DTLS sessions. All software components are considered moderately hardened.

The above architecture generates a visibility graph with 17 nodes, 10 functional edges and 175 implicit ones. The costs correspond to protection level of communication protocols and hardening level of software components, the scale is from 0 to 80 chosen by the analysed system owner experts and discussed with the IT solution suppliers. For example, the cost of breaking HTTPS protocol (resp. COAP, one of the well-known IoT protocols) is estimated at 30 (resp 25), while the cost of malware injection to Linux kernel is estimated at 60. The obtained automata model comprises 17 automata implemented using the UPPAAL [21] model checker.

We studied the system in three phases of its life cycle:

1) (*Design*) with cyber protections choice adapted to the knowledge about their strength at the design phase;
2) (*Production*) when a security flaw has been discovered in the firewall solution leading to update the unitary cost of breaking the firewall protection;
3) (*Mitigation*) when a temporary patch has been applied to the IT architecture, by introducing a modification of the Intranet routing rules.

We considered the following attackers' initial configurations and targets:

1) the attacker ExtHacker coming from Internet, the target being forcing IotFront to produce bad data, for which we used the CTL formula for UPPAAL: `inf{IotFront.B}:costs`
2) the attacker ExtHacker coming from Internet, the target being stealing the secretInfo secret key, for which we used the formula: `inf{StolenSecrets[4]}:costs`,

and two similar ones where the attacker IntHacker comes from Intranet, with analogous formulas. Variable `costs` represents the overall cost of the sequence of moves from the initial configuration to the current one.

The model checking confirmed for the *Design* phase that the minimal costs of attacks were within acceptable limits according to the company policy (fixed to at least 100 for attacks from Internet and 50 for attacks from Intranet), while it was not acceptable for the *Production* phase and it became acceptable again after mitigation. For the *Production* phase, we found for example an attack involving ExtHack, FW and IotFront with overall cost of 60.

For the above system, the model checking computation time has been almost instantaneous on a laptop equipped with Intel Core I9 CPU and 32G RAM.

IV. Outcomes and Limitations

The main outcome of the presented approach is the definition of a model of automata for cyberattack propagation offering satisfactory expressiveness and suitable computational efficiency to cover the main cyber-security protections (*e.g.* authentication, encryption, right restriction, routing restriction, execution environment isolation, informational redundancy) and attack techniques (*e.g.* impersonation, network spoofing, privilege escalation, man-in-the-middle, session hijacking, code injection, fuzzing). Its realisation in UPPAAL [21] allows to automatically discover attacks under cost constraints from

an initial configuration with attackers positioned in some components, leading to targets defined in terms of a set of compromised components.

From the definition of the propagation model (Def. 2) and in particular that of the generic automaton (Def. 4) and its transition conditions and cost calculations (Def. 5), we get the following result (the proof is straightforward):

Proposition 1: Given a setting of an initial configuration with attackers, a target, a definition of unitary costs, and the maximal cost constraint c, the method allows to find an attack in the propagation model whose overall cost is minimal and under c, or prove that such an attack does not exists. □

Among the limitations we face is that our system description only focuses on cyber-security properties in terms of propagation of various forms of compromise but with very little operational functionality aspects. Indeed, component statuses essentially represent a loss of security properties like corruption or non-availability, while an enrichment of the system description by operational properties would have improved the estimation of attack impacts. For example, in the case of our case study, it may be interesting to know whether one of the sensors is under maintenance or is running. These kind of extensions are of course possible, but at the cost of losing computational efficiency and simplicity in terms of system definition.

V. Conclusion and perspectives

Our first attempt of defining a method exploiting the system architecture was [22], where attacks were represented by serial paths in a directed graph, making it difficult and computationally prohibitive to deal with advanced cybersecurity aspects (*e.g.* branching attacks and redundancy). This has been overcome in the current approach thanks to the change of the attack propagation paradigm, providing the needed expressiveness which, combined with realisation in UPPAAL [21], allowed a suitable computational efficiency.

We provide a user-ready software tool which has been used to validate real-life use cases in the context of a collaboration between SystemX and its industrial partners, some examples may be found online [23].

In future work, we plan to explore various monitoring and fallback defence patterns, consider a more advanced cost cumulation method and analyse realistic industrial cases.

Acknowledgement

This work was supported by the French government as part of the "France 2030" program, within the framework of the SystemX Technological Research Institute.

References

[1] National Institute of Standards and Technology. Identifying and estimating cybersecurity risk for enterprise risk management, 2021. https://nvlpubs.nist.gov/nistpubs/ir/2021/NIST.IR.8286A.pdf.

[2] Agence nationale de la sécurité des systèmes d'information. Ebios risk manager, 2019. https://www.ssi.gouv.fr/uploads/2019/11/anssi-guide-ebios_risk_manager-en-v1.0.pdf.

[3] Seema Gupta Bhol, JR Mohanty, and Prasant Kumar Pattnaik. Taxonomy of cyber security metrics to measure strength of cyber security. *Materials Today: Proceedings*, 80:2274–2279, 2023. SI:5 NANO 2021. URL: https://www.sciencedirect.com/science/article/pii/S2214785321046009, doi:10.1016/j.matpr.2021.06.228.

[4] S. Mauw and M. Oostdijk. Foundations of attack trees. In *Information Security and Cryptology-ICISC 2005*, pages 186–198. Springer, 2006.

[5] B. Schneier. Attack trees. *Dr. Dobb's journal*, 24(12):21–29, 1999.

[6] B. Kordy, S. Mauw, S. Radomirović, and P. Schweitzer. Attack–defense trees. *Journal of Logic and Computation*, 24(1):55–87, 06 2012. arXiv:https://academic.oup.com/logcom/article-pdf/24/1/55/3749013/exs029.pdf, doi:10.1093/logcom/exs029.

[7] M. Salfer, H. Schweppe, and C. Eckert. Efficient attack forest construction for automotive on-board networks. In *Information Security*, pages 442–453, Cham, 2014. Springer International Publishing.

[8] M. S. Barik, A. Sengupta, and C. Mazumdar. Attack graph generation and analysis techniques. *Defence Science Journal*, 66(6):559, 2016.

[9] I. Chokshi, N. Ghosh, and S. K. Ghosh. Efficient generation of exploit dependency graph by customized attack modeling technique. In *Advanced Computing and Communications*, pages 39–45. IEEE Computer Society, 2012. URL: https://doi.ieeecomputersociety.org/10.1109/ADCOM.2012.6563582, doi:10.1109/ADCOM.2012.6563582.

[10] H. Singh Lallie, K. Debattista, and J. Bal. A review of attack graph and attack tree visual syntax in cyber security. *Computer Science Review*, 35:100219, 2020. URL: https://www.sciencedirect.com/science/article/pii/S1574013719300772, doi:10.1016/j.cosrev.2019.100219.

[11] K. Kaynar. A taxonomy for attack graph generation and usage in network security. *Journal of Information Security and Applications*, 29:27–56, 2016. URL: https://www.sciencedirect.com/science/article/pii/S2214212616300011, doi:10.1016/j.jisa.2016.02.001.

[12] R.W. Ritchey and P. Ammann. Using model checking to analyze network vulnerabilities. In *IEEE Symposium on Security and Privacy*, pages 156–165, 2000. doi:10.1109/SECPRI.2000.848453.

[13] A. Bagnato, B. Kordy, P. H. Meland, and P. Schweitzer. Attribute decoration of attack–defense trees. *International Journal of Secure Software Engineering*, 2012. doi:10.4018/jsse.2012040101.

[14] Z. Aslanyan and F. Nielson. Pareto efficient solutions of attack-defence trees. volume 9036, pages 95–114, 04 2015. doi:10.1007/978-3-662-46666-7_6.

[15] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J.M. Wing. Automated generation and analysis of attack graphs. In *IEEE Symposium on Security and Privacy*, pages 273–284, 2002. doi:10.1109/SECPRI.2002.1004377.

[16] H. Li, Y. Wang, and Y. Cao. Searching forward complete attack graph generation algorithm based on hypergraph partitioning. *Procedia Computer Science*, 107:27–38, 2017. ICICT2017. URL: https://www.sciencedirect.com/science/article/pii/S1877050917303277, doi:10.1016/j.procs.2017.03.052.

[17] K. Kaynar and F. Sivrikaya. Distributed attack graph generation. *IEEE Transactions on Dependable and Secure Computing*, 13(5):519–532, 2016. doi:10.1109/TDSC.2015.2423682.

[18] K. Piwowarski, K. Ingols, and R. Lippmann. Practical attack graph generation for network defense. In *Computer Security Applications Conference*, pages 121–130. IEEE Computer Society, 2006. URL: https://doi.ieeecomputersociety.org/10.1109/ACSAC.2006.39, doi:10.1109/ACSAC.2006.39.

[19] F. Di Maio, R. Mascherona, and E. Zio. Risk analysis of cyber-physical systems by gtst-mld. *IEEE Systems Journal*, 14(1):1333–1340, 2019.

[20] R. Kumar, S. Schivo, E. Ruijters, B. Mehmet Yildiz, D. Huistra, J. Brandt, A. Rensink, and M. Stoelinga. Effective analysis of attack trees: A model-driven approach. In *FASE 2018*, pages 56–73. Springer International Publishing, 2018.

[21] G. Behrmann, A. David, and K. G. Larsen. A tutorial on UPPAAL. In *LNCS*, volume 3185, pages 200–236. Springer, 2004.

[22] W. Klaudel and A. Rataj. Towards a formalisation of expert's knowledge for an automatic construction of a vulnerability model of a cyberphysical system. In *ICISSP*, pages 391–398. SCITEPRESS, 2021. doi:10.5220/0010207003910398.

[23] SystemX. Automated risk analysis, 2023. https://cti.irt-systemx.fr/ra/.